Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 8: Running SCOPE, part 3

As part of the previous tutorial, you started the submission of three SCOPE tasks (tasks 1-3) for 4 geometries (geom1-4) of 3 systems (water, benzene, formaldehyde). In the best case, these are 36 computations.

As discussed in Tutorial_6, being 3 tasks, you'll need to execute the commands below **at least** 4 times (3+1, see below).

```
scope run -n scope_env_cluster.npy -s water -t task1.scope
task2.scope task3.scope -e
scope run -n scope_env_cluster.npy -s benzene -t task1.scope
task2.scope task3.scope -e
scope run -n scope_env_cluster.npy -s formaldehyde -t task1.scope
task2.scope task3.scope -e
```

(1) The first time you execute this command, SCOPE will submit the computation(s) associated with task1 to the cluster. The System file will be updated and saved

(2) If you execute once task1's computations finish, SCOPE will register task1, and prepare and submit task2. The System file will be updated

Loading [MathJax]/extensions/Safe.js ecute once task2's computations finish, SCOPE will skip task1 (it is already

registered), register task2, and submit task3.
(4) If you execute once task3's computations finish, SCOPE will skip task1 and task2, and register task3. The System file will be updated, and the workflow will be over.

1) Summary of Actions

When executing "scope run":

1. **Navigates the Workflow**: The script locates the Branch and Workflow(s) indicated in the input file, goes through the Jobs and Computations, and takes the appropriate actions:
2. **Submits**: If the job has not been submitted, it creates the input and submission file and does the submission.
3. **Or Waits**: If the job is running, it does nothing
4. **Registers**: If the job has finished, scope *registers* it. That is, it reads the output and decides whether it finished 'well'
   A. If not, it takes appropriate action. For instance:
      a. If it is a GEOMETRY OPTIMIZATION that didn't reach a zero-gradient solution, it creates and submits a new computation that continues the previous run
      b. If it is an SCF computation in Quantum Espresso that didn't converge, it will modify some parameters and resubmit. In Gaussian, convergence failure is rare.
      c. If the computation was aborted by SLURM or for some other reason, the logfile will be discarded, and the computation will be resubmitted.
   B. If the job finished well, the logfile is parsed, and the relevant data is stored in the final state (fstate) defined in the input file. The job will be considered done, and even if you resubmit the same command, the computations won't run again.

2) Actions in Detail

- We will now go through the details: the command "scope run" is just the CLI form to call the function "run_task", which can be found in scope/utils.
- If you open this function with a text editor, you'll identify commented blocks mentioning the STEPs. We'll go through these steps
- To make it easier, we will load the water system (you can load benzene or formaldehyde if you prefer).

IMPORTANT:

- From here, the tutorial assumes that you could run the computations in a cluster ully. At least, task2 should have finished.

Loading [MathJax]/extensions/Safe.js

- If so, you can download the systems generated in your computation cluster. You can update those in the data_folder, or put them in a different place.

Loads Example Case

In [1]:
```
import os
import scope
```

In [2]:
```
## Now we load the System object from a binary file.

## 1- If you updated the old (pre-computations) system files, they are in:
data_folder = os.path.abspath('../')+'/Data/7-Tutorial_7'
path        = f"{data_folder}/Systems/water/water.npy"  # for water, you car

## 2- Otherwise, just insert the path:
#path = ''

## Load the system
sys = scope.load_binary(path)
print(sys)
```

```
---------------------------------
   >>> SCOPE System Object >>>
---------------------------------
 Version                = 1.0
 Type                   = system
 Subtype                = system
 Source Path            = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/water/
 System File Path       = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/
 System File Name       = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/water.npy
 Computations Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/water/

 Num Sources            = 4
     idx: type, name, formula
     0: specie, geom1, H2-O
     1: specie, geom2, H2-O
     2: specie, geom3, H2-O
     3: specie, geom4, H2-O
```

In [3]:
```
## We will discuss the PBE optimization job stored in our example system fil
found, this_branch   = sys.find_branch("tutorial")
found, this_workflow = this_branch.find_workflow("geom1")
found, this_job      = this_workflow.find_job("pbe_opt")
## We will replicate the steps undergone by "scope_run_task" when running th
print(this_job)
```

Loading [MathJax]/extensions/Safe.js

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[3], line 3
      1 ## We will discuss the PBE optimization job stored in our example sy
stem file. First, we navigate the workflow as discussed in Tutorial 2 (Syste
m -> Branch -> Workflow -> Job)
      2 found, this_branch   = sys.find_branch("tutorial")
----> 3 found, this_workflow = this_branch.find_workflow("geom1")
      4 found, this_job      = this_workflow.find_job("pbe_opt")
      5 ## We will replicate the steps undergone by "scope_run_task" when ru
nning this Job.

AttributeError: 'NoneType' object has no attribute 'find_workflow'
```

Step 0:

In [ ]:
```
# In Step 0, the function checks the existence of the files, and loads them.
```

Step 1:

In [ ]:
```
## In Step 1, it initiates the 'ith' LOOP for every task specified by the us
## Scope will iterate through those in order.

## In 1.1, it reads the job_files and complements it with defaults if necess
## In 1.2, it complements the global environment with any user choice specif
## In 1.3, if the environment does not correspond to a computation cluster,
## In 1.4 and 1.5, it finds the requested Branch and the Workflow.
#                  If they do not exist yet, they are created. This is handy
```

Step 2:

In [ ]:
```
## In 2.1, it finds or creates the Job using the "job_data" section in the j
this_job.job_data  ## This is the job_data with which the job was created
```

In [ ]:
```
## In 2.2, if the Job already exists, it checks whether the "job_data" was u
```

In [ ]:
```
## In 2.3, it checks whether the "job" requisites and constrains are fulfill
print(this_job.requisites, this_job.requisites_fulfilled)
print(this_job.constrains, this_job.constrains_fulfilled)

## In this case, both were fulfilled
```

Step 3:

In [ ]:
```
## In Step 3, it finds or creates the computations, using the variable "job_
print(this_job.job_data.job_setup)

## There are 4 options for of job_setup:
##  - "regular" or "reg":          The most common. A Job that expects one c
##  - "replacement" or "disp":     This job expects an initial state (istate
```

Loading [MathJax]/extensions/Safe.js

```
##  - "findiff":                    This job will prepare 3N geometries using
##  - "rep_opt":                    This job will prepare one geometry optimi
```

In [ ]:
```
## In Step 3.1, it checks whether the "qc_data" was updated by the user, and
comp = this_job.computations[0]  ## This is the first and only computation a
print(comp.qc_data)              ## This is its QC_DATA
```

In [ ]:
```
## It also checks whether "continuation" computations exist. This is a very
## The way this is done brings us to the Filename class that we discussed in
print(comp.filename)
## Below are the ITEMS that were used to construct the name of this file. Th
```

In [ ]:
```
## Basically, the Filename class enables SCOPE to identify whether continuat
## A "continuation" computation basically shares the same name of the origin
print(comp.run_number)
## It indicates that it is the first attempt. The run number is the "_r1" pa
print(comp.inp_name)
## In this case, the computation worked at the first attempt, so there was n
print(comp.has_update)
## If otherwise, if a "continuation" computation exists in the calculations
```

In [ ]:
```
## In Step 3.2, it checks whether it must submit the computation. It will do
## (1) The output file does not exist.
## (2) The computation does not have a continuation (i.e. comp.has_update ==
## (3) The user wants to submit computations, as specified in the job_file.
##     The default is True, but reverts to False if SLURM or SGE queue manag
## (4) The job is not running already. This is evaluated by the environment
##     This check can be skipped by the variable "ignore_submitted" in the &

## Also, if the output file exists, but not the input file, the function wil
```

Step 4:

In [ ]:
```
## In step 4, if both input and output files exist, the computation might be
```

In [ ]:
```
## Step 4.1:
## - If the computation is already registered, this step will be skipped. Th
##   This is what would happen in "Case 4" of the example in cell number 3 o

## - If not, the computation is registered (see .Register() function of the
## - Below is a summary of what it does:

## NOTE:
##  From here below, paths might be wrong if you switched computer (local vs
##  If necessary, do the following: comp.set_paths("/path_to_/3-Computations

##   - Files are checked:
print("Input Exists:", comp.input_exists)
print("Stored Path:", comp.inp_path)
##   - The Output-class object is created:
print(comp.create_output())
##   - General Properties are registered
ished:", comp.isfinished)
```

Loading [MathJax]/extensions/Safe.js

```
print("Elapsed Time:", comp.elapsed_time)
print("Status:", comp.status)
```

In [ ]:
```
## Still in Step 4.1:
##   - The Energy is registered and stored in the final state.
found, istate = comp.source.find_state(comp.qc_data.istate)
found, fstate = comp.source.find_state(comp.qc_data.fstate)
print(istate.results["energy"], "in state:", istate.name)
print(fstate.results["energy"], "in state:", fstate.name)
##   - Because this is a job that implies a geometry optimization, the updat
print("Original coordinates of the first atom:", istate.coord[0], "in state:
print("Final coordinates of the first atom:   ", fstate.coord[0], "in state:
```

In [ ]:
```
## In Step 4.2 it sets up continuation computations if needed.
## In Step 4.3 it handles repetitive jobs (only if this_job.job_setup = "rep
this_job.job_setup
```

In [ ]:
```
## In Step 4.4 it handles some errors for which a simple and routine solutic
## This is done only if the flag "-e" is called when executing scope_run_job
```

End

- At the end of the *i-th* loop, if the system has been updated, it is saved (overwritten) where it belongs (the path is taken from *sys.system_file*).
- Once saved, try loading the resulting system files and apply some of the features covered in previous tutorials.
- If you run into any trouble, please contact me at sergi.vela@iqac.csic.es

Loading [MathJax]/extensions/Safe.js